Collision Checking and Nearest Neighbours in C-space

Contents

1. Overview . . . 2
2. Collision detection: broad phase and narrow phase . . . 3
   - 2.1 The yes-or-no question and what it costs . . . 3
   - 2.2 Broad phase and narrow phase . . . 4
   - 2.3 Sweep and prune . . . 4
3. Bounding-volume hierarchies . . . 6
   - 3.1 Why a tree of boxes . . . 6
   - 3.2 Building and searching the tree . . . 6
4. Self-collision . . . 8
5. Distance queries . . . 10
   - 5.1 How far, not only whether . . . 10
   - 5.2 The Minkowski difference and GJK . . . 11
6. Checking a path segment . . . 12
   - 6.1 A segment holds infinitely many configurations . . . 12
   - 6.2 Step size and order . . . 12
   - 6.3 Checking with clearance . . . 14
7. Distances in C-space . . . 15
   - 7.1 The rules a distance must follow . . . 16
   - 7.2 The distance between two headings . . . 16
   - 7.3 Metres and radians in one distance . . . 18
8. Nearest neighbours when headings wrap . . . 18
9. Uniform random rotations and directions . . . 20
   - 9.1 What uniform means for headings . . . 20
   - 9.2 Random directions . . . 21
   - 9.3 Random rotations in 3D . . . 22
10. Summary . . . 24
11. Sources . . . 25
12. Key terms . . . 26

Where this fits:

- **Builds on:** Vector magnitude, distance and scalar operations; Dot product; Uniform distribution; K-nearest neighbours; Minkowski sum; Obstacle region and free space.

1. Overview

Key point: A planner that never builds the obstacle region asks three questions thousands of times: is this configuration free, is this straight segment free, and which stored configuration is nearest? Cheap box tests first, a tree of boxes, well-ordered checks along a segment and a distance that knows headings wrap answer them fast and correctly.

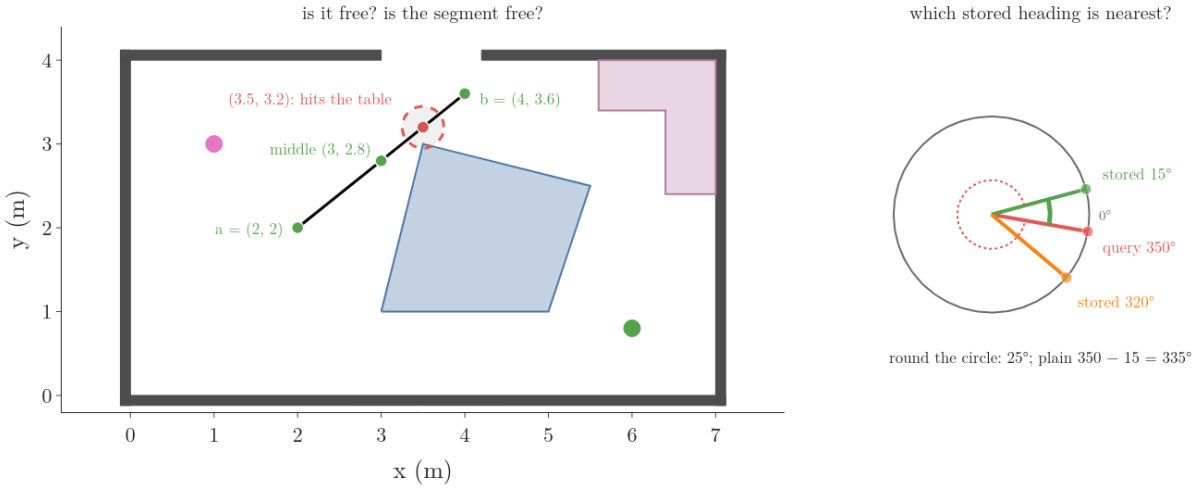


Figure 1: Left: the room with three test points. The robot's disk centred at (3.5, 3.2) (red) overlaps the table. The segment from a = (2, 2) to b = (4, 3.6) has free ends and a free middle (green), yet the point (3.5, 3.2) on it is in collision. Right: a heading dial with a query at 350 degrees and stored headings at 15 and 320 degrees; round the circle 15 is 25 degrees from 350, while plain subtraction says 335

So far the robot knows which of its placements are free. The obstacle region (G-2650), the set of configurations at which the robot's body touches an obstacle, is the room's obstacles grown by the robot's 0.25 m radius. In it, the robot's disk with its centre at (3.5, 3.2) touches the table, and the straight drive from the start (2, 1) to the doorway centre (3.6, 4.0) is free. But building that region exactly is impractical for real robots, so planners ask one question at a time instead (MR 10.2; Hauser ch.10). Three questions come back thousands of times, and on our room each has a trap (Figure 1):

1. **Is this configuration free?** Testing the robot exactly against every obstacle for every configuration is slow: with 10,000 bodies there are 49,995,000 pairs to test (Müller SAP slides).
2. **Is this straight segment free?** A segment holds infinitely many configurations. The segment from (2, 2) to (4, 3.6) is free at both ends and in the middle, yet the robot at

(3.5, 3.2), three quarters of the way along, hits the table.

3. **Which stored configuration is nearest?** Headings wrap round: a stored heading of 15° is 25° from a query at 350°, but plain subtraction says 335°, so a plain search returns the wrong neighbour.

This Note builds the tools that answer them, on the same room and the same 0.25 m disk:

- cheap box tests before exact ones (Section 2), a tree of boxes (Section 3) and links of one robot that may hit each other (Section 4);
- how far a robot is from an obstacle (Section 5) and how to check a whole segment (Section 6);
- a distance for configurations with headings (Section 7), a nearest-neighbour search that knows headings wrap (Section 8), and random configurations that favour no direction (Section 9).

2. Collision detection: broad phase and narrow phase

> **Key point: Collision detection** (G-2665) answers "do these two bodies overlap?" in two steps: the **broad phase** (G-2666) throws out pairs whose simple boxes do not overlap, and the **narrow phase** (G-2667) tests the few pairs left exactly. Sweep and prune finds the overlapping boxes by sorting them along one axis.

2.1 The yes-or-no question and what it costs

The first question of the opening problem: is the configuration with centre (3.5, 3.2) free? Answering it means **collision detection** (G-2665): deciding, yes or no, whether two bodies overlap (Hauser ch.7, "Proximity queries"). Besides the walls, the robot can overlap four pieces of furniture in our room (the table, the cabinet and the two poles), so one configuration needs four exact tests for them, and a planner runs thousands of configurations. The exact test is the costly part: for the disk against the table it means the distance from the centre to a polygon; for detailed bodies, made of thousands of triangles like triangle meshes (G-2338), many more steps.

With n bodies there are this many pairs, because each body pairs with every other body and each pair is counted twice:

- n is the number of bodies: 5 in our room (the robot and the four pieces of furniture).

$$\text{pairs} = \frac{n(n-1)}{2}$$

$$= \frac{5 \times 4}{2} = 10$$

For a scene of 10,000 bodies, the same count gives 49,995,000 pairs (Müller SAP slides). Most of those pairs are far apart and could be ruled out at a glance. The question is how to glance cheaply.

2.2 Broad phase and narrow phase

> **Key point:** Wrap each body in its smallest box with sides along x and y. Two bodies can only touch if their boxes overlap, and two boxes are compared with four number comparisons.

A box round a body is a quick stand-in for it: if two boxes do not overlap, the bodies inside cannot overlap either. Comparing two boxes takes four comparisons of numbers, far fewer steps than an exact test. So collision detection runs in two phases (Müller SAP slides; LaValle §5.3.1):

1. the **broad phase** (G-2666): wrap every body in a simple shape and keep only the pairs whose shapes overlap;
2. the **narrow phase** (G-2667): test the kept pairs exactly.

A simple shape that encloses a body for such a quick test is a **bounding volume** (G-2668): a sphere, a box, or a box turned with the body (Hauser ch.7). The commonest is the smallest box with sides along the x and y axes that holds the body, which we call the body's box. Two such boxes overlap exactly when their x-intervals overlap and their y-intervals overlap (Hauser ch.7).

On our room, with the robot's centre at (3.5, 3.2) (notebook):

Body	x from	x to	y from	y to
robot	3.25	3.75	2.95	3.45
table	3.0	5.5	1.0	3.0
cabinet	5.6	7.0	2.4	4.0
pink pole	0.9	1.1	2.9	3.1
green pole	5.9	6.1	0.7	0.9

Only the table's box overlaps the robot's: their x-intervals share 3.25 to 3.75, and their y-intervals share 2.95 to 3.0. So the broad phase leaves one pair, and the narrow phase tests it exactly: the centre is 0.2 m from the table, less than the 0.25 m radius, so the robot and the table overlap (the centre lies in the grown table). One exact test instead of four. The walls are left out of this comparison for now: a wall's long, thin box lies along a whole side of the room, which Section 3 handles.

2.3 Sweep and prune

Testing the robot's box against every other box is still a test per pair, 10 for our 5 bodies. **Sweep and prune** (G-2670) cuts that number by sorting (Müller SAP slides):

1. sort the boxes by their left edge;
2. sweep a vertical line from left to right: a box joins the list of active boxes when the line reaches its left edge and leaves at its right edge;
3. a box that joins is compared only with the boxes already active, the ones it overlaps along x;

4. for those pairs, a full box test checks y as well; a pair that passes goes to the narrow phase.

Figure 2 runs the sweep on our five boxes. Watch the stage "x = 3.25 m: the robot joins": the table is active, so the robot is compared with it, and the box test flags the pair. At "x = 5.9 m: the green pole joins", the cabinet is active, but their y-intervals, 0.7 to 0.9 and 2.4 to 4.0, do not overlap. The last frame, "narrow phase on the one flagged pair", runs the exact test.

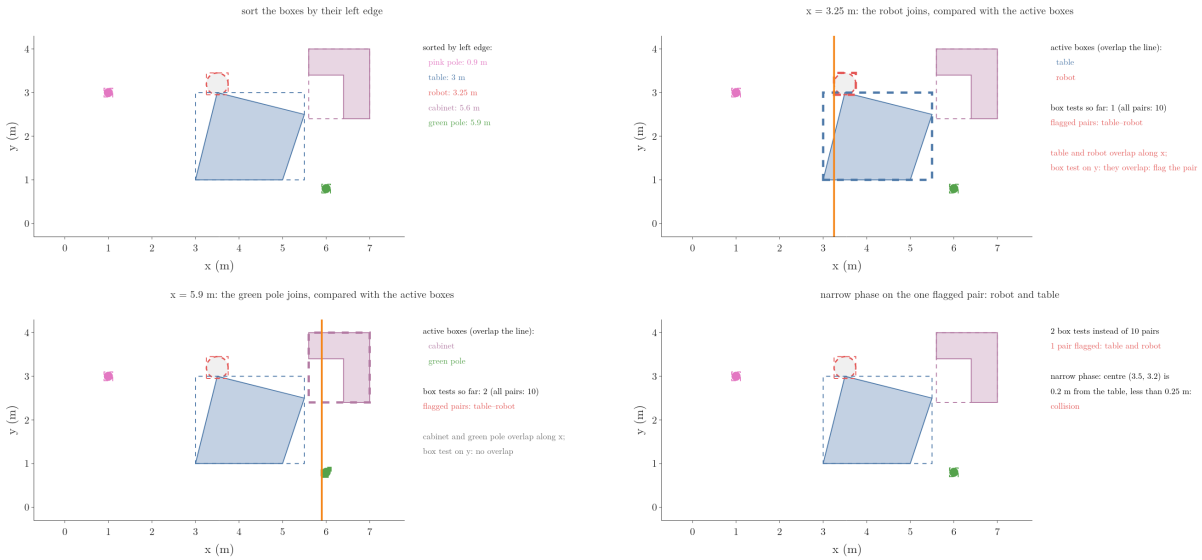


Figure 2: Animation: sweep and prune on the room's five bodies, the robot with its centre at (3.5, 3.2) (red dashed disk) and the table, cabinet and two poles, each inside its dashed box with sides along x and y; walls left out. The boxes are sorted by left edge: pink pole 0.9 m, table 3.0, robot 3.25, cabinet 5.6, green pole 5.9. An orange sweep line moves right; a box joins the active list (thick outline) at its left edge and leaves at its right edge. When the robot joins, the table is active: the boxes overlap along x and along y, so the pair is flagged. When the green pole joins, the cabinet is active: they overlap along x but not along y. The last frame: 2 box tests instead of 10 pairs; the narrow phase finds the robot 0.2 m from the table, less than 0.25 m: a collision

Sorting puts the boxes that might overlap next to each other, so each box looks only at its neighbours along x. The sort takes far fewer steps than testing all pairs (Müller SAP slides). With:

- n the number of bodies, each in its box, now 10,000;
- $\log_2 n$ the number of times n can be halved before reaching 1, 13.3 for 10,000.

$$\text{sort steps} \approx n \log_2 n$$

$$\log_2 10{,}000 = 13.3$$

$$10{,}000 \times 13.3 = 133{,}000$$

The sort's 133,000 steps are far below the 49,995,000 pairs, though how many pairs the sweep then compares depends on how the boxes lie (Müller SAP slides). Sweep and prune looks along one axis only, and a long wall's box overlaps almost every other box along that axis.

3. Bounding-volume hierarchies

> **Key point:** A **bounding-volume hierarchy** (G-2671) is a tree of boxes: each box holds two smaller ones, down to one box per body. A query opens a box only if it overlaps the robot's box, so for n bodies it reaches the few near the robot after about $\log_2 n$ levels: 9 box tests among the room's 25 bodies.

3.1 Why a tree of boxes

The walls were left out of the sweep. Put them in, and each wall's box stretches along a whole side of the room, overlapping almost every box along x, so sorting along one axis separates little (Müller BVH slides). With many bodies spread in two or three dimensions the problem grows: for 100×100 objects in a plane, sweep and prune still tests about 100 objects for each one (Müller BVH slides).

The fix is to group nearby bodies, and groups of groups, each group in one box. A query starts at the box that holds everything; if the robot's box misses a group's box, every body inside is ruled out at once. A tree of nested bounding volumes, each holding the boxes of its two children down to one box per body, is a **bounding-volume hierarchy** (G-2671), BVH for short (Hauser ch.7; Müller BVH slides).

So that each wall takes up only a small box, we store it as panels about 1 m long, as a triangle mesh (G-2338) stores a surface in small pieces: 7 panels along the bottom, 4 up each side and 3 on each side of the doorway, 21 in all. With the 4 pieces of furniture, the room has 25 bodies.

3.2 Building and searching the tree

The tree is built from the top down (Müller BVH slides; Hauser ch.7):

1. one box round all 25 bodies is the root;
2. sort the bodies of a box by the centres of their boxes along x and split them into two halves of equal count (13 and 12 here); each half gets its own box;
3. repeat inside each half, sorting along y, then x again, and so on, until each box holds one body.

A query for the robot's box descends from the root (Hauser ch.7, "BVH queries"):

1. test the box of the current node against the robot's box;
2. if they miss, skip the node and everything inside it;

3. if they overlap and the node holds one body, pass that body to the narrow phase; otherwise test both children.

Figure 3 builds the tree level by level, then runs the query for the robot at (3.5, 3.2). Watch the room on the left and the tree on the right: a red box is a miss, and every body under it is skipped without a test. The stage "query, test 4" is the first miss: the box of the room's lower-left walls, which the robot's box does not reach. The last frame shows the 9 tests that leave one body, the table.

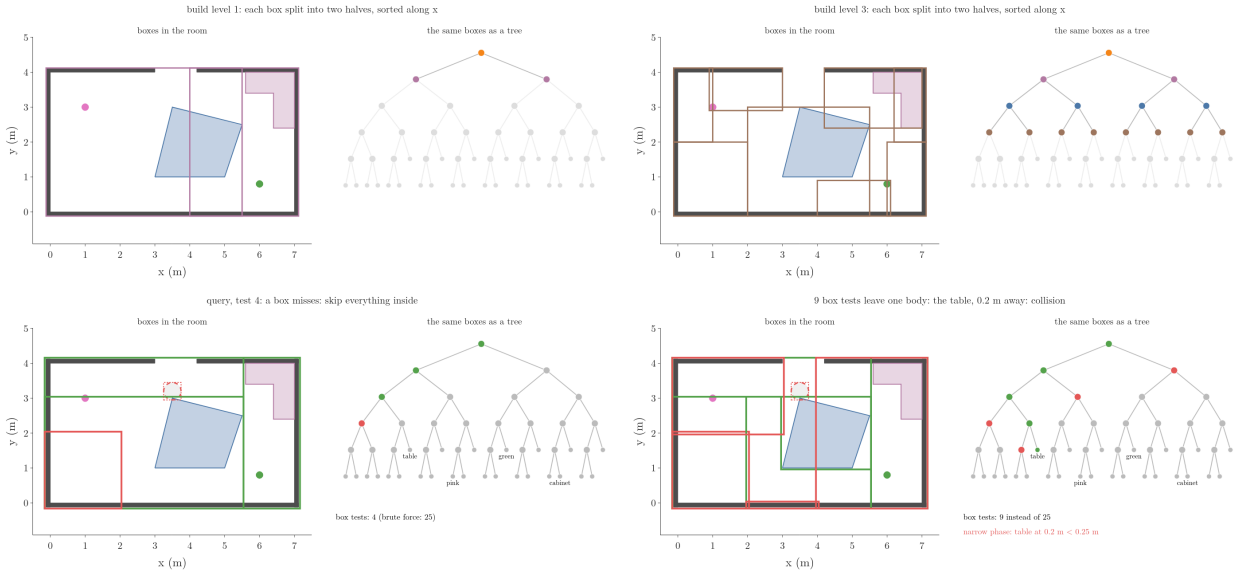


Figure 3: Animation: a bounding-volume hierarchy over the room's 25 bodies (4 pieces of furniture and 21 wall panels about 1 m long). Build: one box round everything, then each box split into two halves of equal count, sorting along x, then y, then x, coloured by level, on the room (left) and as a tree (right). Query for the robot with its centre at (3.5, 3.2), its box dotted red: each tested box turns green if it overlaps the robot's box and red if it misses; a red box's whole subtree is skipped. 9 box tests reach the table, the only body left for the narrow phase, 0.2 m from the robot's centre: a collision. Brute force would test 25 boxes

The query needed 9 box tests instead of 25 (notebook). From the start (2, 1), where nothing is close, it needs 11, and over 2,000 random centres in the room it averages 12.0. The tree has 5 levels below the root, because halving 25 bodies 4 times still leaves boxes with two bodies, and 5 times leaves one:

$$2^4 = 16 < 25 \leq 32 = 2^5$$

In general, for a balanced tree:

- n is the number of bodies, such as 32;
- L is the number of levels below the root, 5 for 32 bodies.

$$n = 2^L$$

$$L = \log_2 n$$

so a query that goes down one branch makes about $\log_2 n$ box tests (Müller BVH slides). With 10,000 objects in a plane that is about 13 tests per object, against about 100 for sweep and prune and 10,000 for brute force (13.3 from Section 2.3); with 1,000,000 objects in 3D, about 20 against about 10,000 and 1,000,000 (Müller BVH slides):

$$\log_2 1{,}000{,}000 = 19.9$$

The count grows when the robot is close to many bodies at once, since then several branches must be opened (Hauser ch.7, "Performance"). A BVH is built once for bodies that do not move and reused for every query, which is why planners use it for the room's furniture and walls. The robot itself, though, may be more than one body.

4. Self-collision

> **Key point:** A robot made of several links can hit itself. A **self-collision** (G-2672) check tests the pairs of links that can meet; links joined by a joint touch all the time and are left out.

The first question, is this configuration free, has one more part for robots with several links. Our disk robot is one rigid body, so it cannot hit itself. An arm can: its links move relative to each other, and a far link can fold back onto a near one. We switch to a new example for this one section: a planar arm with three links, 1.0 m, 0.8 m and 0.6 m long, drawn as line segments, like the 2R arm, the arm with two joints on the robot's top plate, with a third link added. Its configuration is three joint angles, each measured from the direction of the link before it.

Figure 4 shows two configurations. At joint angles (0°, 90°, 90°) the arm folds into a hook and link 3 stays clear of link 1. At (0°, 150°, 150°) link 3 folds back across link 1, crossing it at (0.54, 0): a **self-collision** (G-2672), the robot's own links overlapping (LaValle §4.3.1, "Obstacle region for multiple bodies").

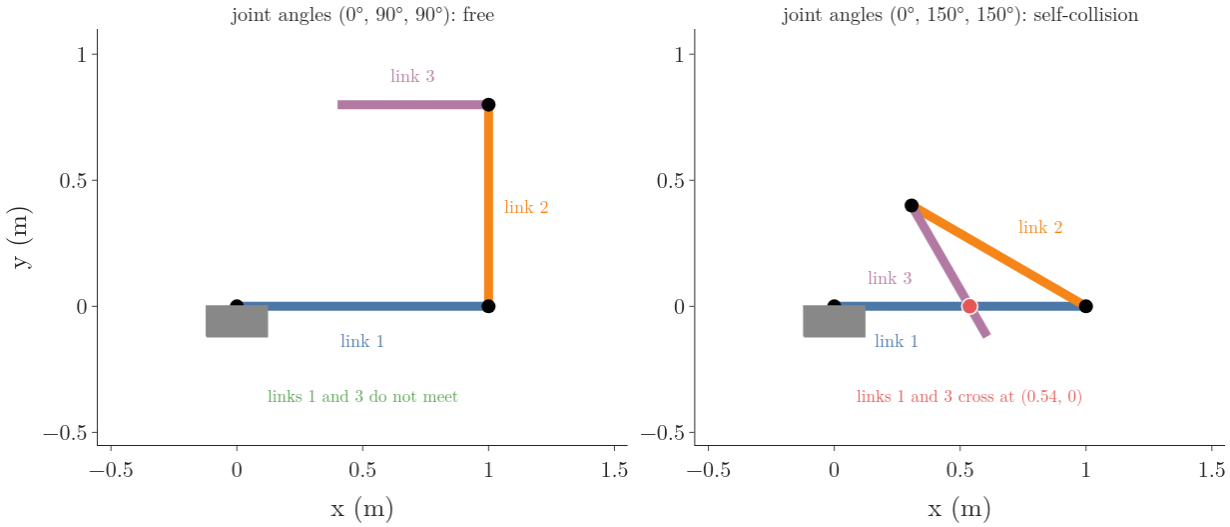


Figure 4: A planar arm with three links (1.0 m blue, 0.8 m orange, 0.6 m purple) on a base at the origin. Left: joint angles (0, 90, 90) degrees; links 1 and 3 do not meet. Right: joint angles (0, 150, 150) degrees; link 3 folds back across link 1 and crosses it at (0.54, 0), marked red: a self-collision

Which pairs to test. The pairs to test are the pairs of links that must not overlap (LaValle §4.3.1). Links next to each other share a joint, so they touch all the time and are left out (LaValle §4.3.1). For a chain of m links, the count of pairs is the one we used for bodies:

$$\text{all pairs} = \frac{m(m-1)}{2}$$

$$\text{pairs joined by a joint} = m - 1$$

For our arm, with 3 links:

$$\frac{3 \times 2}{2} - 2 = 1 \text{ pair: links 1 and 3}$$

For an arm with 7 links:

$$\frac{7 \times 6}{2} - 6 = 15 \text{ pairs}$$

Some of the remaining pairs can never meet whatever the joint angles, and some always overlap; testing them wastes time. So robot software such as MoveIt's Setup Assistant samples 10,000 random configurations by default and turns off checking for the pairs that are never in collision, always in collision, in collision in the robot's default pose, or joined by a

joint (MoveIt docs). For our arm, links 1 and 3 cross in 735 of 10,000 random configurations, 7.35 percent (notebook), so the pair stays on the list.

The self-collision pairs join the broad phase and the narrow phase like any other pair: a configuration is free only if no link hits an obstacle and no pair on the list overlaps (LaValle §4.3.1, eq. 4.36). Every check so far answers yes or no; a planner often needs to know by how much.

5. Distance queries

> **Key point:** A **distance query** (G-2674) returns how far apart two bodies are, not only whether they touch. For two convex bodies it equals the distance from the origin to their Minkowski difference, which the **GJK algorithm** (G-2678) finds from the shapes' farthest points in chosen directions, without building the difference.

5.1 How far, not only whether

The second question, whole segments, needs more than yes or no. At the start (2, 1) the robot is free, but how much room does it have? A planner uses the answer to move fast where there is room and carefully near obstacles, and Section 6 uses it to clear whole segments at once. A **distance query** (G-2674) returns the shortest distance between two bodies, 0 if they overlap (Hauser ch.7, "Proximity queries"). The distance from the robot's body to the nearest obstacle is its clearance, the room it has to move.

Figure 5 (left) answers it for the start. The robot's centre is 1.0 m from the table's corner (3.0, 1.0) and 1.0 m above the bottom wall, and the body reaches 0.25 m of that:

$$1.0 - 0.25 = 0.75 \text{ m}$$

So the clearance at the start is 0.75 m, to the table and to the bottom wall alike (notebook). Seen in C-space, the clearance is the distance from the centre to the nearest grown obstacle (G-2652). When two bodies overlap, a distance query can say instead how deep the overlap is: at (3.5, 3.2) the disk reaches 0.05 m into the table (Hauser ch.7).

5.2 The Minkowski difference and GJK

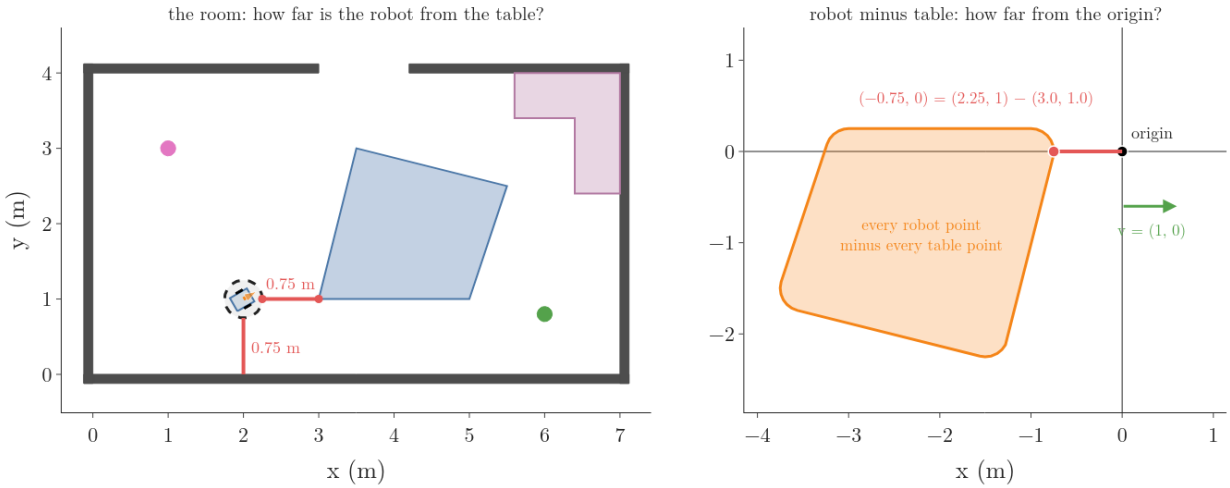


Figure 5: Left: the room; the robot at the start (2, 1) is 0.75 m from the table's corner (3.0, 1.0) and 0.75 m from the bottom wall (red). Right: the Minkowski difference robot minus table (orange), every robot point minus every table point; it does not contain the origin, and its nearest point to the origin is (−0.75, 0), 0.75 m away. That point is the robot's farthest point in the direction v = (1, 0), (2.25, 1), minus the table's farthest point in the direction −v, (3.0, 1.0)

For a disk the distance was one subtraction. For two bodies of any convex shape, the Minkowski difference (G-2654), every point of one shape minus every point of the other, gives a single rule. Take every point of the robot minus every point of the table:

- if the two share a point, some difference is $(0, 0)$, so the difference contains the origin and the bodies overlap;
- otherwise the distance between the bodies is the smallest length of a difference, which is the distance from the origin to the Minkowski difference (dyn4j GJK distance).

In Figure 5 (right), the orange shape is the robot at (2, 1) minus the table. The shape does not contain the origin, and its nearest point to the origin is (−0.75, 0): 0.75 m, the clearance found above.

Farthest points. Building the whole difference costs as much as building the obstacle region. The **GJK algorithm** (G-2678), named after Gilbert, Johnson and Keerthi, never builds it (dyn4j GJK). GJK works with one point at a time: the point of a shape farthest in a chosen direction, found as the point with the largest dot product (G-634) with that direction, each coordinate times the direction's, added (dyn4j GJK). With:

- v the chosen direction, $(1, 0)$, from the robot towards the table;
- $-v$ the opposite direction, $(-1, 0)$.

The dot products of the table's four corners (3, 1), (5, 1), (5.5, 2.5) and (3.5, 3) with $-v$ are:

$$-3.0,\ -5.0,\ -5.5,\ -3.5$$

The largest is -3.0, so the table's farthest point in the direction $-v$ is the corner (3.0, 1.0). The robot's farthest point in the direction v is its rim point straight ahead of the centre:

$$(2 + 0.25,\ 1) = (2.25,\ 1)$$

The farthest point of the difference in the direction v is the robot's farthest point in v minus the table's farthest point in $-v$ (dyn4j GJK), because the dot product of a difference with v is the robot point's dot product minus the table point's, which is largest when the first is largest and the second smallest:

$$(2.25,\ 1) - (3.0,\ 1.0) = (-0.75,\ 0)$$

The point $(-0.75, 0)$ is already the nearest point of the difference to the origin. In general GJK picks a direction towards the origin, adds the farthest point found there, and repeats with the triangle (in 3D, the tetrahedron) of points found so far, until a triangle encloses the origin (overlap) or no new point gets past the origin (apart); each step costs one farthest point per shape (dyn4j GJK). A bounding-volume hierarchy speeds up distance queries as it speeds up yes-or-no checks, as in the collision libraries PQP and FCL (Hauser ch.7). With distances in hand, the second question of the opening problem, whole segments, can be answered.

6. Checking a path segment

> **Key point:** A segment cannot be checked point by point, so we check points at most a step apart: too large a step misses thin collisions, and checking in halving order (ends, middle, quarter points, ...) finds a collision sooner. With clearances, a piece of segment is free once the free circles at its two ends meet.

6.1 A segment holds infinitely many configurations

The second question of the opening problem: is the straight segment from a = (2, 2) to b = (4, 3.6) free? Both ends are free and so is its middle, (3, 2.8). A planner cannot test every point, because a segment holds infinitely many configurations (LaValle §5.3.4), so it tests some points and hopes the rest are free. Checking a segment by testing points along it is **edge resolution** (G-2679), also called dynamic collision checking (Hauser ch.10).

The segment is 2.561 m long. The notebook finds its colliding stretch: from 0.620 to 0.777 of the way along, 0.403 m round the table's corner, (3.5, 3.2) among it.

6.2 Step size and order

Step size. Split the segment into k equal steps no longer than a chosen step ε (Hauser ch.10, "Dynamic collision checking"):

- $d(a, b)$ is the segment's length, 2.561 m;
- ε is the longest step allowed, in m;
- $\lceil \cdot \rceil$ rounds up to a whole number;
- k is the number of steps; the checks are its $k + 1$ end points.

$$k = \left\lceil \frac{d(a, b)}{\varepsilon} \right\rceil$$

With $\varepsilon = 0.6$ m:

$$k = \lceil 2.561/0.6 \rceil = \lceil 4.27 \rceil = 5$$

$$\text{step} = 2.561/5 = 0.512 \text{ m}$$

The checks sit at 0, 0.2, 0.4, 0.6, 0.8 and 1 of the way. The colliding stretch runs from 0.620 to 0.777, between the checks at 0.6 and 0.8, so all six checks are free and the segment wrongly passes. A step shorter than the thinnest obstacle a collision can hide in is needed; too short a step wastes time (Hauser ch.10; LaValle §5.3.4). With $\varepsilon = 0.1$ m we halve the segment until each piece is shorter than 0.1 m, so that the halving order below can use the same points:

$$2.561 \to 1.281 \to 0.640 \to 0.320$$

$$\to 0.160 \to 0.080 \text{ m}$$

Five halvings give 32 steps of 0.080 m, so 33 points.

Order. Checking those 33 points from a towards b, the first collision is check 21, at 0.625 of the way. Checking them in halving order instead (both ends, then the middle point, then the two quarter points, then the eighths, and so on), the fifth check, three quarters of the way, is the point (3.5, 3.2), and it hits (LaValle §5.3.4). When both ends are free, the point most likely to be in collision is the one farthest from both, the middle (Hauser ch.10); the halving order keeps checking the points farthest from those already checked, so its early checks spread over the whole segment and a collision anywhere is found soon (LaValle §5.3.4). If the segment is free, every point must be checked in either order, so the order costs nothing then (LaValle §5.3.4).

Figure 6 runs the three checks. Watch the stage "5 even steps: every check is free, yet the segment is not", then the long row of checks from a until the hit, then "the same 33 points in halving order", which hits at the fifth check.

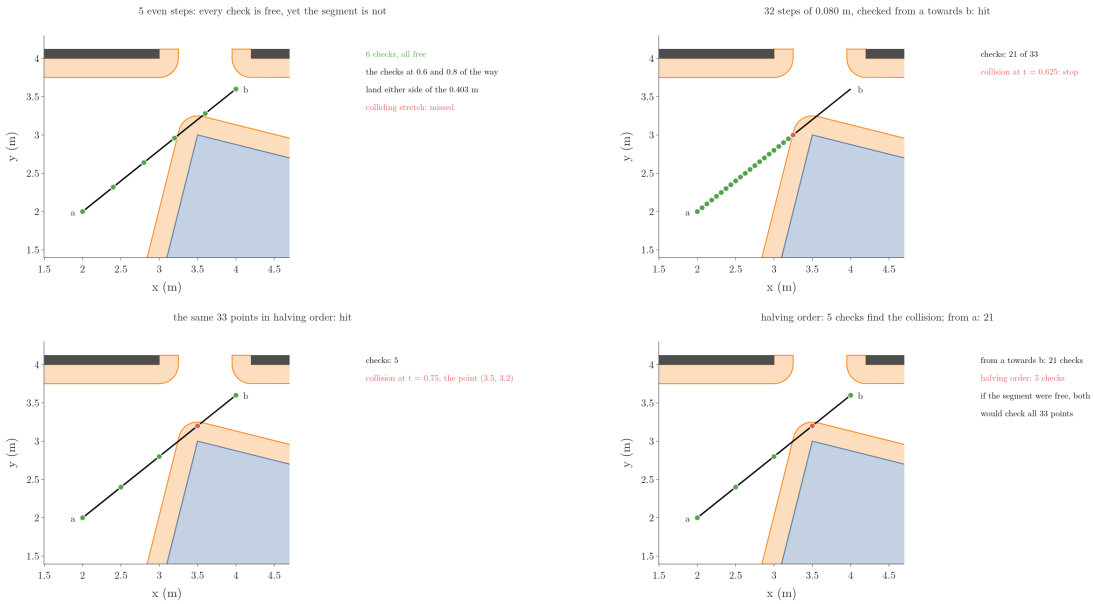


Figure 6: Animation: checking the segment from a = (2, 2) to b = (4, 3.6) in C-space, with the obstacles grown by 0.25 m (orange). Part 1: 5 even steps of 0.512 m; the 6 checks (green) are all free, because the checks at 0.6 and 0.8 of the way land on either side of the 0.403 m colliding stretch. Part 2: 32 steps of 0.080 m checked from a towards b; the 21st check, at 0.625 of the way, is the first in collision (red). Part 3: the same 33 points in halving order (ends, middle, quarter points); the 5th check, at 0.75 of the way, the point (3.5, 3.2), is in collision. Last frame: 5 checks instead of 21

6.3 Checking with clearance

A step of 0.1 m is a guess: a thinner obstacle could still hide between two checks. Clearances remove the guess (Hauser ch.10). Around a configuration with clearance c, the robot can move up to c in any direction without touching anything, so the circle of radius c round it is free. If the free circles at the two ends of a piece of segment reach each other, the piece is covered and free. With:

- a and b the two ends of a piece of segment;
- $d(a, b)$ its length, in m;
- $c(a)$ and $c(b)$ the clearances at its ends, in m.

$$d(a, b) \leq c(a) + c(b)$$

If the length passes this test, the piece is free. Otherwise, check its middle and repeat on both halves (Hauser ch.10, “Visible-Exact1”; KAIST proximity slides). For the drive from the start to the doorway, free with 0.132 m to spare, the first test fails:

$$3.4 > 0.75 + 0.35 = 1.1$$

so the middle is checked, and so on. Figure 7 draws each check with its free circle. Watch the circles shrink as the checks approach the table's corner and grow again towards the door. The check nearest the corner, at (3.2, 3.25), has a clearance of 0.141 m; the drive's closest point to the grown table, 0.132 m, lies between two checks. In the last frame, "the last circles meet", nine circles cover the whole drive.

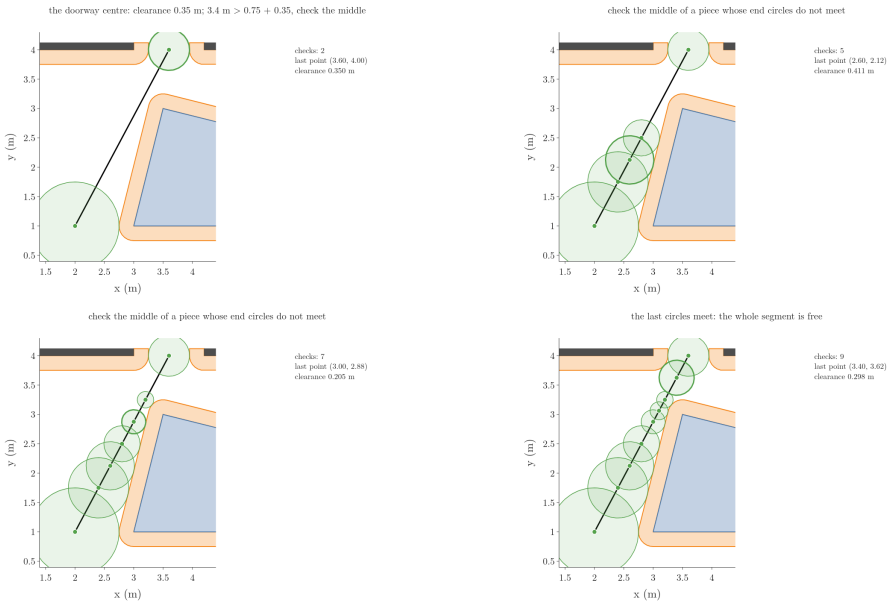


Figure 7: Animation: checking the drive from the start (2, 1) to the doorway centre (3.6, 4.0) with clearances. Each checked point (green) gets a circle whose radius is its clearance from the grown obstacles (orange): 0.75 m at the start, 0.35 m at the doorway centre. A piece whose end circles do not meet gets its middle checked. The circles shrink near the table's corner (0.141 m at (3.2, 3.25)) and grow towards the door; after 9 checks the circles cover the whole 3.4 m drive, so it is free

Nine distance queries prove the 3.4 m drive free (notebook), where checks every 0.1 m would take 35 collision checks and would still only sample it. A distance query costs more than a yes-or-no check, which is why planners mostly use the step method (Hauser ch.10; LaValle §5.3.4). Both answer the first two questions; the third needs a distance between configurations, not between bodies.

7. Distances in C-space

> **Key point:** A **metric** (G-2681) is a distance that is never negative, is 0 only between equal points, is the same both ways, and obeys the **triangle inequality** (G-2683): a detour through a third point is never shorter. For headings the **angular distance** (G-2684) goes the short way round the circle, and metres and radians are combined with a weight.

7.1 The rules a distance must follow

The third question of the opening problem: which stored configuration is nearest? "Nearest" needs a distance between two configurations. The Euclidean distance (G-715), the straight-line distance, is one, and the nearest neighbours (G-1306) of a point are the stored points at the smallest distance from it. For any distance function a planner relies on, four rules must hold (LaValle §5.1.1; KAIST C-space slides). With:

- p, q and r three configurations, such as the headings 350°, 15° and 0°;
- $d(p, q)$ the distance from p to q.

The rules, each followed by its formula:

1. never negative;
2. zero only for the same configuration;
3. the same both ways;
4. the **triangle inequality** (G-2683): going through r cannot be shorter than going straight.

$$d(p, q) \geq 0$$

$$d(p, q) = 0 \text{ only if } p = q$$

$$d(p, q) = d(q, p)$$

$$d(p, q) \leq d(p, r) + d(r, q)$$

A function that obeys all four is a **metric** (G-2681) (LaValle §5.1.1). The Euclidean distance is a metric, and so is the Manhattan distance, the sum of the absolute differences; both belong to the Minkowski distance (G-1227) family (KAIST C-space slides). Why the rules matter: the k-d tree search of Section 8 skips a whole box when the box is farther than the best point found so far, which is only safe if no detour is shorter than the straight distance (Hauser ch.10, "k-d trees"; Wikipedia, "Metric tree").

7.2 The distance between two headings

Headings break the plain distance. The query at 350° and the stored 15° of Figure 1:

$$|350 - 15| = 335°$$

but turning the other way, across 0°, is 25°. Worse, 0° and 360° are the same heading, yet:

$$|0 - 360| = 360°$$

so plain subtraction breaks rule 2: two equal headings at a distance of 360°. Angle wrapping (G-2404) fixes it: add or subtract full turns until the difference lies between −180° and 180°. Its size is the **angular distance** (G-2684), the length of the short way round the circle (LaValle §5.1.2, eq. 5.7; Hauser ch.4, "Geodesic distance and interpolation"). With headings θ_1 and θ_2 in degrees:

$$d = \min(|\theta_1 - \theta_2|,\ 360 - |\theta_1 - \theta_2|)$$

For 350° and 15°:

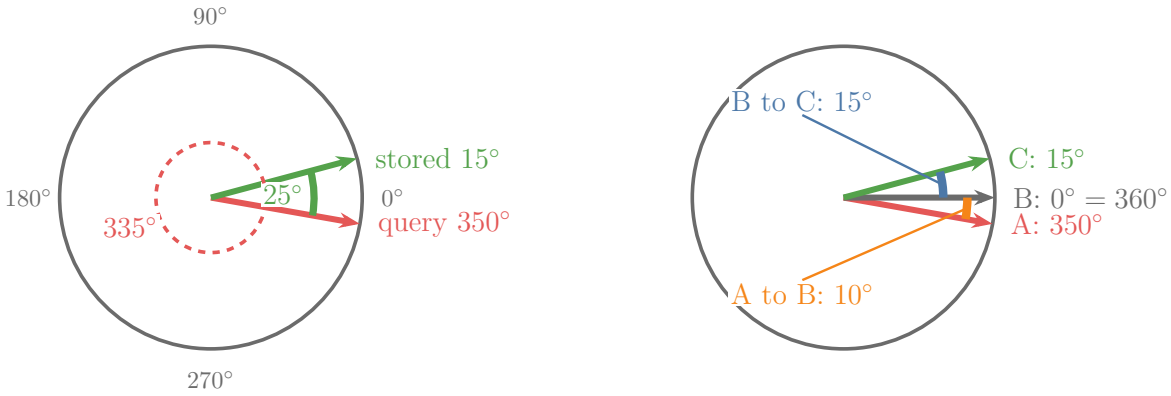
$$d = \min(335,\ 360 - 335) = 25°$$

Figure 8 (left) shows the two ways round. The angular distance obeys the four rules (LaValle §5.1.2): on the right of Figure 8, A at 350°, B at 0° and C at 15°, and the detour through B is no shorter:

$$d(A, C) = 25$$

$$d(A, B) + d(B, C) = 10 + 15 = 25$$

$$25 \leq 25$$



plain subtraction: $350 - 15 = 335°$
the short way round: $360 - 335 = 25°$

A to C through B: $10 + 15 = 25°$
A to C directly: 25°, no longer than through B

Figure 8: Two heading dials. Left: a query at 350 degrees and a stored heading at 15 degrees; plain subtraction goes the long way, 350 − 15 = 335 degrees (red, dashed), the short way round across 0 is 360 − 335 = 25 degrees (green). Right: the triangle inequality on the dial: A at 350, B at 0 = 360, C at 15 degrees; A to C through B is 10 + 15 = 25 degrees, and A to C directly is 25 degrees, no longer than through B

7.3 Metres and radians in one distance

A configuration of the floor robot has a position in metres and a heading in radians, and a distance must combine them. One radian of turn moves the rim of the 0.25 m footprint 0.25 m (the arc rule of radians (G-2285)), so we count 0.25 m for each radian of angular distance (LaValle §5.3.4, eq. 5.30; KAIST C-space slides). A longer robot, such as one carrying a 1.4 m board, would need a larger weight: a small turn moves its far end a lot (LaValle §5.3.4). With:

- $q_1 = (x_1, y_1, \theta_1)$ and $q_2 = (x_2, y_2, \theta_2)$ two configurations;
- $\Delta\theta$ the angular distance between θ_1 and θ_2, in rad;
- w the weight, 0.25 m per rad;
- $d(q_1, q_2)$ their distance, in m.

$$d(q_1, q_2) = \sqrt{\Delta x^2 + \Delta y^2 + (w\,\Delta\theta)^2}$$

where Δx and Δy are the differences in x and y (KAIST C-space slides). The weighted distance is a metric: it is the Euclidean distance on positions and on scaled headings measured the short way (LaValle §5.1.1, "Cartesian products of metric spaces"). For the query at x = 3.55 m, 350°, and the stored configuration E at x = 3.60 m, 15° (y equal), the heading part is 25°, which is 0.436 rad:

$$0.25 \times 0.436 = 0.109 \text{ m}$$

$$d = \sqrt{0.05^2 + 0.109^2} = 0.120 \text{ m}$$

With plain subtraction, 335° would make the heading part 1.462 m and E look far. A distance that wraps is half of the answer; the search must wrap too.

8. Nearest neighbours when headings wrap

> **Key point:** A k-d tree search skips a cell when the cell is farther than the best point found so far. If headings are treated as numbers from 0 to 360, a cell just across 0° looks a long way off and is skipped; measuring the cell's distance the short way round fixes the search.

The opening problem's third trap, in a search tree. A k-d tree stores points in cells split one coordinate at a time, so a nearest-neighbour search can skip every cell that lies farther away than the best point found so far (Hauser ch.10, "k-d trees"; LaValle §5.5.2). Figure 9 stores 8 configurations in one, with two coordinates so that it fits a page, x and heading; y would work like x. The tree splits on x, then heading, then x. The query is at x = 3.55 m, heading 350°. Distances use the weight of Section 7.3, 0.25 m per radian, and the heading axis is drawn in those metres, so a circle of equal distance is round.

Plain search. The search goes first into the cell that holds the query (Hauser ch.10). The search finds C and then D, at 0.371 m, and then reaches the cell of headings 0° to 175° beside it. Measured as plain numbers, that cell is 175° below the query:

$$350 - 175 = 175^\circ = 3.054 \text{ rad}$$

$$0.25 \times 3.054 = 0.764 \text{ m}$$

The gap of 0.764 m is more than the best so far, 0.371 m, so the cell is skipped, and with it E. The search ends at H (3.40 m, 320°), 0.199 m away. Wrong: E is nearer, at 0.120 m (Section 7.3).

Search with wrap-around. The heading 0° is the same as 360°, so the cell of headings 0° to 175° starts 10° above the query, across the top edge:

$$360 - 350 = 10^\circ = 0.175 \text{ rad}$$

$$0.25 \times 0.175 = 0.044 \text{ m}$$

Less than 0.371 m, so the cell is visited, and E, at 0.120 m, becomes the best. Measuring a cell's distance the short way round the circle, and its points' distances with the angular distance, is all that changes; we call this a **nearest-neighbour search with wrap-around** (G-2685). LaValle §5.5.2 warns that the tree search must respect the places where the space wraps round, and Hauser ch.10 notes that headings need their own point-to-cell distance.

Figure 9 runs both searches. Watch the stage "plain search, step 3", where the cell holding E is pruned as 0.764 m away, then "with wrap-around search, step 3", where the same cell is 0.044 m away and E is found. The circle of the best distance so far wraps from the top edge to the bottom.

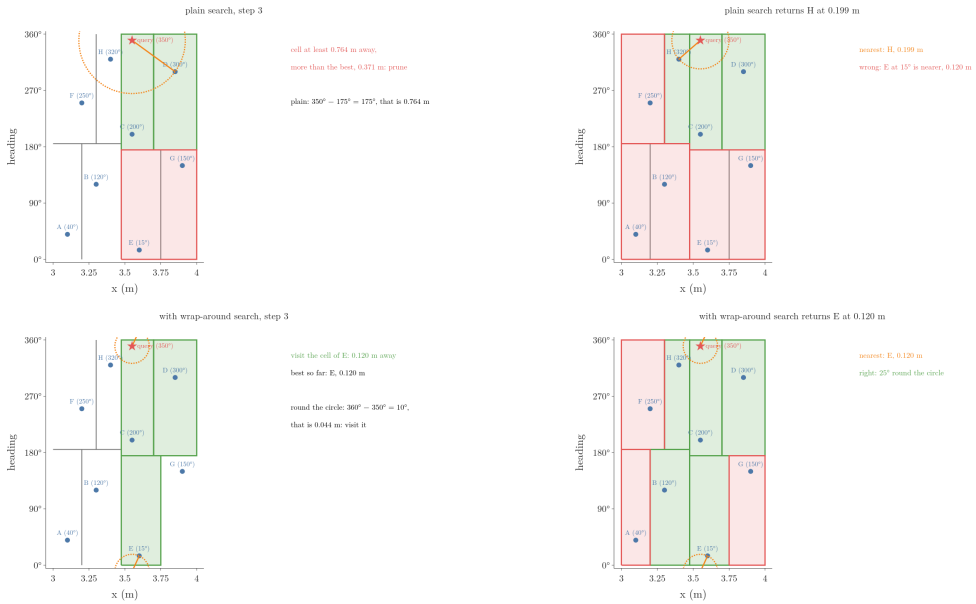


Figure 9: Animation: nearest-neighbour search in a k-d tree over 8 stored configurations A to H (x in m across, heading up, the heading axis drawn at 0.25 m per radian so circles are round), for the query at x = 3.55 m, 350 degrees (red star). Visited cells turn green, pruned cells red; the dotted orange circle is the best distance so far. Plain search visits C, then D (0.371 m), prunes the cell of headings 0 to 175 degrees as 0.764 m away (350 − 175 = 175 degrees), and returns H at 0.199 m. Search with wrap-around measures that cell the short way, 360 − 350 = 10 degrees, 0.044 m, visits it and returns E at 0.120 m; its circle wraps from the top edge to the bottom

A planner that grows a tree of configurations asks this question at every step (Hauser ch.10, "Nearest-neighbor queries"), so the wrap-around must be right every time. The configurations it adds come from random sampling, which has a trap of its own: random orientations that favour some directions.

9. Uniform random rotations and directions

> **Key point:** A random heading must favour no direction: drawn evenly from 0° to 360°, every arc of 30° gets the same share, 1/12, wherever it lies. Directions and rotations in 3D need care: a uniform latitude angle crowds the poles, a point in a cube favours the corners; a normal sample scaled to length 1, the bread-slicer recipe and Shoemake's recipe for 3D rotations are uniform.

9.1 What uniform means for headings

Sampling planners draw random configurations, so the random headings must not favour any direction. For the floor robot this is easy. Draw the heading from the continuous uniform distribution (G-467) from 0° to 360°, every value equally likely. Then any arc of 30° gets the

same share of the draws, wherever on the circle it lies (notebook, 100,000 draws: 8.4 percent for 10° to 40°, 8.3 percent for 200° to 230°):

$$30/360 = 0.083$$

Equal shares for equal arcs, wherever they lie, is what uniform means for rotations. A **uniform random rotation** (G-2686) makes every orientation equally likely: turning the whole set of samples by any fixed rotation must not make some orientations more common (LaValle §5.1.4, §5.2.2). The circle of headings has no favoured place, so the plain uniform draw is right. Directions and rotations in 3D, as for a drone or the gripper of an arm, are where the traps are.

9.2 Random directions

A **uniform random direction** (G-2689) is a unit vector (a vector of length 1) drawn so that every direction is equally likely: every patch of the circle, or of the sphere, of the same size gets the same share (LaValle §5.2.2).

Two traps and two fixes, with the shares from the notebook (Figure 10):

- **A point in a square, scaled to length 1.** Draw x and y evenly from -1 to 1 and divide by the length. The square reaches 1.41 times as far along a diagonal as along an axis, so a thin wedge along a diagonal holds about twice the area, and diagonal directions come up more often: within 5° of a diagonal 16.2 percent of the time, within 5° of an axis 8.6 percent, against a fair share of 11.1 percent (LaValle §5.2.2 warns of this bias towards a cube's corners).
- **A normal sample, scaled to length 1.** Draw each coordinate from a normal distribution with the same spread. Its density depends only on the distance from the centre, so every direction is equally likely: 11.1 percent near the diagonals and 11.1 percent near the axes (LaValle §5.2.2).
- **On the sphere, a uniform latitude angle.** Drawing the latitude evenly from $-90°$ to 90° packs as many points into the short circles near the poles as into the long equator. The two caps above height 0.9 and below -0.9 hold 10 percent of the sphere's surface but get 28.7 percent of the points.
- **On the sphere, a uniform height: the bread slicer.** Drawing the height evenly from -1 to 1 and the longitude evenly from 0° to 360° is uniform, because slices of a sphere of equal thickness have equal crust area, like a loaf through a bread slicer (LaValle §5.2.2). The caps get 10.1 percent.

The caps by the two recipes, the height 0.9 being at latitude 64.2°:

$$\arcsin 0.9 = 64.2°$$

$$\text{latitude: } 2 \times (90 - 64.2)/180 = 0.287$$

$$\text{height: } 2 \times (1 - 0.9)/2 = 0.1$$

The latitude recipe gives each degree of latitude the same share; the height recipe gives each slice of height the same share, which is right because equal slices have equal area.



Figure 10: Top: 100,000 random directions in the plane as a rose of 10-degree bins (bar length: percent of directions in the bin). Left, a point drawn evenly in the square from -1 to 1 and scaled to length 1: bumps on the diagonals, 16.2 percent within 5 degrees of a diagonal against 8.6 percent near an axis. Right, a normal sample scaled to length 1: even, 11.1 percent each, the fair share. Bottom: 1,500 points on the sphere from three samplers, orange where the height is above 0.9 or below -0.9 (two caps with 10 percent of the surface). Uniform latitude angle: 28.7 percent in the caps. Uniform height (bread slicer): 10.1 percent. The up arrow turned by Shoemake's random rotations: 10.0 percent

9.3 Random rotations in 3D

A rotation in 3D, such as the orientation of a gripper, can be stored as four numbers $h = (h_1, h_2, h_3, h_4)$ whose squares add up to 1, so h is a point on a sphere in four dimensions (LaValle §5.2.2):

$$h_1^2 + h_2^2 + h_3^2 + h_4^2 = 1$$

Uniform rotations are uniform points on that sphere, and the bread-slicer idea carries over one dimension up. Shoemake's recipe (LaValle §5.2.2, eq. 5.15), with:

- u_1, u_2, u_3 three numbers drawn evenly from 0 to 1;
- h the four random numbers that describe the rotation.

$$h = (\sqrt{1-u_1}\sin 2\pi u_2,$$

$$\sqrt{1-u_1}\cos 2\pi u_2,$$

$$\sqrt{u_1}\sin 2\pi u_3,$$

$$\sqrt{u_1}\cos 2\pi u_3)$$

Check that the squares add up to 1. For $u_1 = 0.3$, $u_2 = 0.6$ and $u_3 = 0.9$ the recipe gives $(-0.492, -0.677, -0.322, 0.443)$ (notebook). The squares of the first two entries, then of the last two:

$$0.242 + 0.458 = 0.700 = 1 - 0.3$$

$$0.104 + 0.196 = 0.300 = 0.3$$

$$0.700 + 0.300 = 1$$

The same steps in symbols, with $s = 2\pi u_2$, the first two entries squared:

$$(1-u_1)\sin^2 s + (1-u_1)\cos^2 s$$

$$= (1-u_1)(\sin^2 s + \cos^2 s)$$

$$= 1 - u_1$$

The last two give u_1 the same way, and together:

$$(1-u_1) + u_1 = 1$$

Check that it is uniform. Turn the arrow pointing straight up by 20,000 of these rotations. If every rotation is equally likely, the arrow's tip lands evenly on the sphere, so the caps above height 0.9 should get 10 percent. The caps get 10.0 percent (notebook; Figure 10, bottom right). Drawing three angles evenly (turns about z, y and x, each from 0° to 360°) and turning the arrow puts 6.5 percent there, not 10: three uniform angles do not make a uniform rotation (notebook), as LaValle §5.2.2 warns.

10. Summary

Tool	Answers	On our room
Broad phase and narrow phase	is this configuration free? cheaply	1 exact test instead of 4 at (3.5, 3.2)
Sweep and prune	which boxes overlap?	2 box tests instead of 10 pairs
Bounding-volume hierarchy	which bodies are near the robot?	9 box tests among 25 bodies
Self-collision pairs	which links can hit each other?	1 pair for 3 links, 15 for 7
Distance query, GJK	how far apart?	clearance 0.75 m at the start
Edge resolution	is this segment free?	halving order: 5 checks instead of 21
Angular distance and weighted metric	how far apart are two configurations?	350° to 15°: 25°, not 335°
Search with wrap-around	which stored configuration is nearest?	E at 0.120 m, not H at 0.199 m
Uniform rotations and directions	a random orientation that favours none	caps get 10.0 percent, not 28.7

- The three questions of the opening problem are answered on our room: (3.5, 3.2) is in collision, found with one exact test after the boxes, because only the table's box overlaps the robot's; the segment from (2, 2) to (4, 3.6) is not free, found at the fifth check in halving order, because the early checks spread evenly over the segment; and the nearest stored configuration to 350° is E at 15°, 0.120 m away, because the search measures headings the short way round.
- **Collision detection** (G-2665) splits into a **broad phase** (G-2666) with boxes whose sides run along x and y and a **narrow phase** (G-2667), so the costly exact test runs only on pairs whose boxes overlap.
- **Sweep and prune** (G-2670) sorts the boxes along x, so each box is compared only with boxes it overlaps along x, but long boxes such as walls defeat a single axis.
- A **bounding-volume hierarchy** (G-2671) needs about $\log_2 n$ box tests per query for n bodies, because a box that misses rules out everything inside it: about 13 tests for 10,000 bodies.
- **Self-collision** (G-2672) checks only the pairs of links not joined by a joint, because joined links touch all the time.
- A **distance query** (G-2674) gives the clearance, the room to move; for convex bodies it is the distance from the origin to their Minkowski difference, which **GJK** (G-2678) finds from the shapes' farthest points in chosen directions.
- **Edge resolution** (G-2679) checks points of a segment; too long a step misses the 0.403 m collision, and checking the middle point first, then the quarter points, finds it at check 5 instead of 21; with clearances, 9 checks prove the drive to the door free.
- A **metric** (G-2681) must obey four rules, including the **triangle inequality** (G-2683),

because search trees prune on them; the **angular distance** (G-2684) and a weight of 0.25 m per radian give a metric for configurations with headings.
- A **nearest-neighbour search with wrap-around** (G-2685) measures cells the short way round, so the cell across 0° is 0.044 m away, not 0.764 m, and the right neighbour is found.
- A **uniform random rotation** (G-2686) makes every orientation equally likely; a normal sample scaled to length 1, the bread slicer and Shoemake's recipe are uniform, because they favour no direction, while a uniform latitude crowds the poles.

The robot's planner can now test configurations and segments quickly, measure how far configurations are from each other, find the nearest stored one and draw random configurations evenly. Still missing: a planner that puts these together, drawing random configurations, joining each to its nearest neighbours with checked segments, and growing a route to the door: the sampling-based planners of a later chapter.

11. Sources

Built from

- Ten Minute Physics, "Broad phase collision detection with Sweep and Prune", YouTube, https://www.youtube.com/watch?v=MKeWXBgEGxQ, and its slides https://matthias-research.github.io/pages/tenMinutePhysics/23-SAP.pdf (Müller SAP slides)
- Ten Minute Physics, "Bounding Volume Hierarchies", YouTube, https://www.youtube.com/watch?v=L and its slides https://matthias-research.github.io/pages/tenMinutePhysics/24-morton.pdf (Müller BVH slides)
- Steven M. LaValle, "Motion Planning: Life in C-Space" (ICRA tutorial), YouTube, 1:14:33–1:23:16, https://www.youtube.com/watch?v=ERaZJru80bM
- Reducible, "A Strange But Elegant Approach to a Surprisingly Hard Problem (GJK Algorithm)", YouTube, 0:00–16:38, https://www.youtube.com/watch?v=ajv46BSqcK4
- dyn4j, "GJK (Gilbert-Johnson-Keerthi)", 2010, https://dyn4j.org/2010/04/gjk-gilbert-johnson-keerthi/ (dyn4j GJK), and "GJK, Distance, Closest Points", 2010, https://dyn4j.org/2010/04/gjk-distance-closest-points/ (dyn4j GJK distance)
- KAIST CS686, "CS686: 5. Configuration II", YouTube, 26:01–31:37, https://www.youtube.com/watch and the course's slides "Configuration Space II", https://sgvr.kaist.ac.kr/~sungeui/MPA_F19/Slides/L (KAIST C-space slides), and "Proximity Queries", https://sgvr.kaist.ac.kr/~sungeui/MPA_F19/Slides/ (KAIST proximity slides)
- Hauser, K. *Robotic Systems*, University of Illinois. Ch.4 "3D Rotations": "Geodesic distance and interpolation", https://motion.cs.illinois.edu/RoboticSystems/3DRotations.html; ch.7 "Representing Geometry": "Proximity queries", "Bounding volume hierarchies", "Broad phase collision detection", https://motion.cs.illinois.edu/RoboticSystems/Geometry.html; ch.10 "Motion Planning in Higher Dimensions": "Implicit C-obstacle representation", "Dynamic collision checking", "Nearest-neighbor queries", "k-d trees", https://motion.cs.illinois.edu/RoboticSystems/MotionPlanningHigherDimensions.html (Hauser ch.4, ch.7, ch.10)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §4.3.1 "Obstacle region for multiple bodies" (eq. 4.36), https://lavalle.pl/planning/node159.html;

§5.1.1 “Metric spaces” and “Cartesian products of metric spaces”, https://lavalle.pl/planning/node184.h and https://lavalle.pl/planning/node187.html; §5.1.2 “Important metric spaces for motion planning” (eq. 5.7), https://lavalle.pl/planning/node188.html; §5.1.4 “Using the correct measure”, https://lavalle.pl/planning/node191.html; §5.2.2 “Generating a random element of SO(3)” (eq. 5.15) and “Generating random directions”, https://lavalle.pl/planning/node198.html and https://lavalle.pl/planning/node199.html; §5.3.1 “Two-phase collision detection”, https://lavalle.pl/planning/node214.html; §5.3.4 “Checking a path segment”, https://lavalle.pl/planning/node217.html; §5.5.2 “Approximate solutions” (Kd-trees), https://lavalle.pl/planning/node234.html (LaValle §4.3.1, §5.1.1, §5.1.2, §5.1.4, §5.2.2, §5.3.1, §5.3.4, §5.5.2)

- MoveIt documentation, “MoveIt Setup Assistant”, Step 2: Generate Self-Collision Matrix, https://moveit.picknik.ai/main/doc/examples/setup_assistant/setup_assistant_tutorial.html (MoveIt docs)

Other references

- Northwestern Robotics, “Modern Robotics, Chapter 10.2: C-Space Obstacles”, YouTube, https://www.youtube.com/watch?v=s2qrMwqm4D0
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.2.1 “Configuration Space Obstacles” and §10.2.2 “Distance to Obstacles and Collision Detection”. Free preprint: http://modernrobotics.org (MR 10.2)
- Hauser, K. *Robotic Systems*, ch.9 “Geometric Motion Planning”, https://motion.cs.illinois.edu/Robotic
- Wikipedia, “Metric tree” (search trees that use the triangle inequality to skip candidates), https://en.wikipedia.org/wiki/Metric_tree (Wikipedia, “Metric tree”)

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Collision detection (G-2665)	Deciding, yes or no, whether two bodies overlap, such as the robot’s disk and the table; planners call it thousands of times instead of building the obstacle region.
Broad phase (G-2666)	The first, cheap step of collision detection: wrap each body in a simple shape such as a box and keep only the pairs whose shapes overlap, so the costly exact test runs on few pairs.

Term	Meaning
Narrow phase (G-2667)	The second step of collision detection: the exact overlap test on the pairs the broad phase kept, such as the distance from the robot's centre to the table compared with its 0.25 m radius.
Bounding volume (G-2668)	A simple shape that encloses a body, such as a sphere or a box; if two bounding volumes do not overlap, neither do the bodies, so it gives a quick test before an exact one.
Sweep and prune (G-2670)	A broad phase that sorts the boxes by their left edge and sweeps along x, comparing each box only with the boxes it overlaps along x; it cuts the pairs tested, though long boxes that span the axis defeat it.
Bounding-volume hierarchy (BVH) (G-2671)	A tree of nested bounding volumes, each holding its two children's down to one per body; a query skips every body inside a box that misses, so it needs about $\log_2 n$ box tests for n bodies (9 among the room's 25).
Self-collision (G-2672)	Two links of the same robot overlapping, such as an arm's third link folding back across its first; a configuration with a self-collision is not free, so planners test the robot's own link pairs too.
Distance query (G-2674)	A geometric query that returns the shortest distance between two bodies, 0 if they overlap; it tells a planner how much room the robot has, not only whether it is free.
GJK algorithm (G-2678)	The Gilbert-Johnson-Keerthi algorithm: it decides whether two convex shapes overlap, or how far apart they are, by searching their Minkowski difference for the origin using each shape's farthest point in chosen directions, without building the difference.
Edge resolution (segment checking) (G-2679)	Checking a straight path segment in C-space for collisions by testing points along it, spaced at most a chosen step apart; too long a step can miss a thin collision, too short a step wastes time.

Term	Meaning
Metric (distance function) (G-2681)	A distance between points that is never negative, is 0 only between equal points, is the same both ways and obeys the triangle inequality; search trees rely on these rules to skip far regions safely.
Triangle inequality (G-2683)	The rule that a detour through a third point is never shorter than going straight, $d(p, q) \leq d(p, r) + d(r, q)$; it is what lets a nearest-neighbour search skip a region farther than its best point.
Angular distance (G-2684)	The distance between two headings measured the short way round the circle, the smaller of $\lvert\theta_1 - \theta_2\rvert$ and 360° minus it, so 350° and 15° are 25° apart, not 335°.
Nearest-neighbour search with wrap-around (G-2685)	A k-d tree search that measures a cell's distance from the query the short way round the circle of headings, so a cell just across 0° is not skipped and the true nearest configuration is found.
Uniform random rotation (G-2686)	A random rotation drawn so that every orientation is equally likely: turning the whole set of samples by any fixed rotation does not make some orientations more common; for headings, a draw evenly from 0° to 360°.
Uniform random direction (G-2689)	A random unit vector drawn so that every direction is equally likely; a normal sample scaled to length 1 gives one, while a point in a square or cube scaled to length 1 favours the diagonals.
Obstacle region (C-obstacle) (G-2650)	The set of all configurations at which the robot's body touches or overlaps an obstacle, C_{obs}; drawing it in C-space lets a planner test a placement as one point, such as the disk robot's centre (3.5, 3.2), which lies in it 0.2 m from the table.
Triangle mesh (G-2338)	A set of triangles, each given by three corner points in space, that together cover an object's surface (a table as 12 triangles on 8 corners); fast to draw in 3D, but it does not say what is inside.

Term	Meaning
Grown obstacle (inflated obstacle) (G-2652)	An obstacle with every point within the robot's radius added, its edges pushed out and its corners rounded; for a disk robot it is exactly the obstacle region, so the robot can be treated as a point (the table, 3.625 m^2, grows to 5.747 m^2 for a 0.25 m radius).
Minkowski difference (G-2654)	The set of all differences of a point of one shape and a point of the other, $A \ominus B$, which equals $A \oplus (-B)$; obstacle minus robot gives the obstacle region, and two convex bodies overlap exactly when their difference contains the origin.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^\mathsf{T}x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Euclidean distance (G-715)	The straight-line distance between two points.
Nearest neighbours (neighbours) (G-1306)	The observations at the smallest distance from a given point; in KNN, the k training points closest to the query point, whose values or labels decide the prediction.
Minkowski distance (G-1227)	A family of distances between two points, $(\sum_i \lvert x_i - y_i \rvert^p)^{1/p}$, set by the number p: $p = 2$ gives the Euclidean distance and $p = 1$ the Manhattan distance; KNN uses it to find the nearest neighbours.
Angle wrapping (G-2404)	Adding or subtracting full turns until an angle or angle difference lies between $-180°$ and $180°$, so that $175°$ and $-178°$ count as $7°$ apart, not $353°$; needed whenever bearings or headings are compared.
Radian (G-2285)	The unit of angle at which the arc of a circle is as long as its radius; a full turn is 2π rad, and with radians an arc is simply radius times angle.
Continuous uniform distribution $U(a, b)$ (G-467)	A continuous variable spread evenly between a and b, with density $1/(b - a)$.